Riadenie Autonomneho vozidlav simulátore Carla využitím DQN a PPO

1st Martin Buzogan
Katedra počítačov a informatiky
Technická univerzita v Košiciach
Košice, Slovakia
martin.buzogan.2@student.tuke.sk

2nd Alexander Bodnár
Katedra počítačov a informatiky
Technická univerzita v Košiciach
Košice, Slovakia
alexander.bodnar@student.tuke.sk

3th Matuš Petro
Katedra počítačov a informatiky
Technická univerzita v Košiciach
Košice, Slovakia
matus.petro.2@student.tuke.sk

***Abstract*—Naša práca sa zaoberá vývojom Reinforcement Learning agentov (DQN a PPO) pre riadenie autonómneho vozidla v simulátore CARLA. Hlavným cieľom je naučiť agenta bezpečne a efektívne navigovať v simulovanej dopravnej premávke s rôznymi dynamickými podmienkami, minimalizovať riziko kolízií a adaptovať sa neočakývaným situáciám. Implementovali sme dve prístupy strojového učenia: Deep Q-Network (DQN), ktorý využíva hodnotové funkcie, a Proximal Policy Optimization (PPO), ktorý optimalizuje politiku. Na trénovanie agentov sme využili vizuálne vstupy zo segmentačnej kamery a vlastné OpenAI Gymnasium prostredie pre simulátor CARLA. Práca demonštruje možnosti a výzvy využitia RL agentov v oblasti autonómnej jazdy v realistických simulovaných podmienkach.**

***Index Terms*—autonomous driving**

I. Úvod

Cieľom tejto práce je navrhnúť, implementovať a porovnať agentov využívajúcich Deep Q-Network (DQN) a Proximal Policy Optimization (PPO) pre autonómne riadenie vozidla v premávke. Pri výbere metód sme sa zamerali na porovnanie DQN a PPO algoritmov, ktoré patria medzi najrozšírenejšie RL algoritmy v oblasti autonómneho riadenia. V nasledujúcich častiach práce podrobne popíšeme použité metódy, implementáciu vlastného prostredia v simulátore CARLA, prístup trénovania jednotlivých modelov a diskuziu dosiahnutých výsledkov.

A. Význam Reinforcement Learningu v autonómnej mobilite

Reinforcement Learning zohráva dôležitú úlohu v oblasti autonómnej mobility, pretože umožňuje agentom učiť sa optimálne stratégie priamo zo spätnej väzby získanej z prostredia. Princíp pokusu a omylu, na ktorom je založený celý RL, zabezpečuje zlepšovanie a prispôsobenie agenta na neznáme alebo neočakávané situácie. Tento typ učenia RL sa bežne využíva pre vývoj autonómnych vozidiel, schopných spoľahlivo reagovať na komplexné a rýchlo sa meniace dopravné scenáre. V kontexte autonómnych vozidiel je využitie RL veľmi výhodné, pretože dopravné prostredie je vysoko dynamické, nelineárne a často nepredvídateľné. Klasické algoritmy majú obmedzené možnosti reagovať na neočakávané udalosti alebo nové scenáre, ktoré neboli explicitne zohľadnené pri návrhu a realizácií riešenia. RL práve tento nedostatok týchto klasických algoritmov obchádza. Umožňuje agentovi vytvárať generalizované stratégie na základe skúseností z rôznych situácií, čo je nevyhnutné pre bezpečné a efektívne riadenie vozidla v reálnych podmienkach.

B. Použité metódy Reinforcement Learningu

V rámci nášho zadania sa zameriame na metódy Reinforcement Learningu, ktoré pokryjú odlišné prístupy. Jednej už spomenutých algoritmov na princípe hodnotových funkciách (DQN) a druhej na priamo optimalizovanej politike (PPO).

1) Deep Q-Network (DQN): Deep Q-Network je rozšírením klasického Q-learningu pre prostredia s veľkým alebo spojitým stavovým priestorom.

- **Princíp:** Agent sa učí aproximovať akčnú hodnotovú funkciu $Q(s, a)$ pomocou neurónovej siete $Q_\theta(s, a)$.
- **Sieťová architektúra:** Vstupom sú reprezentácie stavov (obrázky), následne niekoľko skrytých plne prepojených alebo konvolučných vrstiev, a na výstupe je vektor, ktorý má rozmer rovný veľkosti akčného priestoru.
- **Replay buffer:** Agent ukladá prechody (s, a, r, s') do pamäte a natrénuje sa na náhodných minibatchoch, čím sa znižuje korelácia dát a zvyšuje stabilita učenia.
- **Cieľová sieť:** Pre stabilnejšie učenie využívame cieľovú sieť so staršími parametrami θ^-, ktorá sa periodicky aktualizuje z aktuálnej siete.
- **Aktualizačné pravidlo:** Minimalizujeme Bellmanovu chybu

$$L(\theta) = \mathbb{E}_{(s,a,r,s')}\Big[\big(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\big)^2\Big].$$

2) Proximal Policy Optimization (PPO): Proximal Policy Optimization patrí medzi moderné metódy priamo optimalizujúce politiku bez potreby explicitnej hodnotovej aproximácie.

- **Princíp:** Agent priamo optimalizuje parametrizovanú politiku $\pi_\theta(a|s)$ pomocou gradientov tak, aby maximalizoval očakávanú kumulatívnu odmenu. kde $\hat{A}_t$ je odhad výhody (advantage).

- **Generalized Advantage Estimation (GAE):** Na zníženie rozptylu odhadov používame GAE, ktoré hladko kombinuje n-krokové návraty.
- **Batch aktualizácie:** PPO spravidla vykonáva viacero gradientných krokov na rovnakom dátovom batchi, pričom používa relatívne malé kroky, aby sa zachovala stabilita učenia.
- **Clipped objektív:** Kľúčovou vlastnosťou PPO je použitie tzv. „clipped“ objektívnej funkcie, ktorá obmedzuje príliš veľké zmeny v politike medzi aktualizáciami. Matematicky sa optimalizuje nasledujúci výraz:

$$L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r(\theta)A,\ \mathrm{clip}(r(\theta), 1-\varepsilon, 1+\varepsilon)A\right)\right]$$

Po úspešnej implementácii oboch prístupov budeme porovnávať ich výsledky z hľadiska:

1) **Dĺžka trénovania:** Budeme sledovať, koľko epizód je potrebných na to, aby jednotlivé modely dosiahli stabilný výkon – teda konvergenciu učenia. Porovnáme rýchlosť učenia oboch prístupov tým, že vyhodnotíme, ktorý z agentov dosiahne požadovanú úroveň bezpečnej jazdy v kratšom čase a s menším počtom trénovacích epizód.
2) **Počet kolízií:** Bezpečnosť jazdy budeme hodnotiť na základe počtu kolízií, ku ktorým dôjde počas testovacích jázd po natrénovaní modelov. Budeme sledovať celkový počet kolízií aj priemerný počet kolízií na epizódu v rôznych testovacích scenároch.
3) **Schopnosť generalizovať:** Overíme, ako sa agenti prispôsobujú novým, neznámym alebo zmeneným podmienkam prostredia ako je jazda v hustej alebo riedkej premávke, či dokonca na nových trasách, na ktorých agenti trénovaní neboli. Týmto spôsobom vyhodnotíme univerzálnosť oboch prístupov.

C. Vytvorenie vlastného prostredia pre simulátor CARLA

Pri riešení nášho RL prostredia sme využili knižnicu OpenAI Gymnasium s realistickým simulátorom CARLA, ktorý je postavený na Unreal Engine. Rozhodli sme sa využiť map 'Town02' ,ktorá je znázornená na fig. n.1.

Fig. 1. Znázornenie trénovacej mapy

Pre zmniernenie hardwaroevých nárokov vďaka jej kompaktnosti, taktiež nás zaujala jednoduchšou cestnou infraštruktúrov kde sa nachádza iba jeden pruh pre zvládnutie smeru .

Na ovládanie prostredia sme využili Gymnasium wrapper s Python API simulátora, ktorý zabezpečuje inicializáciu a ovládanie simulácie – metódy reset() a step(action) poskytujú agentovi jednotné rozhranie vďaka čomu agent môže plnohodnotne ovládať simulátor CARLA a návrhnúť správne odmeňovacie stratégie a tréningové postupy. Stavový priestor agenta tvorí vizuálna informácia vo forme segmentačnej fotografie. Agent „vidí“ rôzne objekty pomocou rôzných farbieb ako su znázornené na fig. n.2 . Obraz zo segmentačnej kamery s rozmermom (128×256x3) je následne spracovaný konvolučnými vrstvami. Tento formát vstupných dát umožňuje agentovi neustále vyhodnocovať okolitú situáciu.

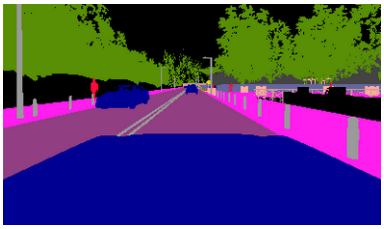

Fig. 2. Znázornenie výstupu segmentačnej kamery

Simulačné prostredie v Carle funguje na architektúre Server-Klient. Simulátor základne pracuje na asynchrónnom móde, kedy server tzv. "nečaká"na klienta v našom prípade vozidlo, ktoré sníma okolité prostredie. Takéto riešenie môže spôsobiť nekonštantné snímanie prostredia pri náročnejšej scéne. Simulátor obsahuje aj tzv. synchrónny mód, kedy server "čaká" na klienta kedy je pripravený pokračovať. Vďaka tomu je možné zabezpečiť aby náš ahent zikaval konštantne snímky prostredia. Práve pri tomto riešení je možné regulovane hýbať s vozidlom. Server bude nútený počkať na klienta, kým server nebude vyzvaný našim klientom.

Tento synchróny mód podporuje nastavenie rýchlosti simulácie prostredníctvom parametra fixed_delta_seconds. Tento parameter určuje dlžku jedného simulačného kroku v sekundách. Inak povedané časový interval, o ktorý sa simulácia posunie pri každom zavolaní klientom. Počas experimentovania sme testovali viacero hodnôt tohto parametra, aby sme zistili, ktorá z nich je najvhodnejšia pre trénovanie RL. Skúšali sme nižšie aj vyššie hodnoty, čo priamo ovplyvňovalo plynulosť simulácie, rýchlosť učenia aj náročnosť na výpočtové zdroje.

TABLE I
NASTAVENIA SIMULÁTORA CARLA

synchronous_mode	True
fixed_delta_second	0.05

Po porovnaní výsledkov sme sa rozhodli pre hodnotu fixed_delta_seconds = 0.05 Táto hodnota znamená, že simulácia beží efektívne na 20 FPS. Takže pri tomto nastavení simulátor vykonáva 20 aktualizácií za sekundu.

II. Trénovacia fázy pre agentov RL

A. Model DQN

Pre prvotný tréning modelu DQN bol použitý diskrétny akčný priestor so základnými príkazmi: pridať plyn, zabrzdiť a natočiť volant doľava či doprava. To ale nebolo dostačujúce, pretože agent pri tréningu robil prudké akcie. Rozšírením diskrétneho priestoru sme zabezpečili jemnejšie a presnejšie akcie vykonávané agentom.

Úvodná fáza trénovania DQN agenta bola zameraná na prvotné vnímanie okolitého prostredia. To znamená udržanie sa na vozovke, zabránenie kolíziám s objektami na okraji vozovky, vnímenie stredu jazdného pruhu. Nevyhnutnou akciou v trénovaní bolo agenta naučiť sa pohybovať v smere jazdy.

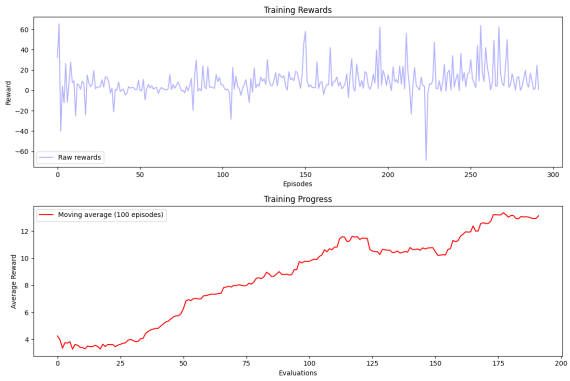

Fig. 3. Graf 1. fázy trénovania

Sekundárna fáza kládla dôraz na udržiavanie jazdy v jazdnom pruhu a správne prechádzanie zákrut a križovatiek, kde sme sa snažili zapríčiniť prechodom do protismerov. Problémom pri tejto fáze bolo, že návratové reward hodnoty sa nesprávali správne v zákrutách a križovatách ako na rovných úsekoch cesty. Toto nerovnomerné oceňovanie agenta viedlo k nesprávnemu učeniu. Riešením bolo oddeliť reward funkciu na funkciu, ktorá hodnotí agenta, keď sa nachádza v križovatkách a zákrutách a funkciu ktorá hodnotí agenta na rovných cestách.

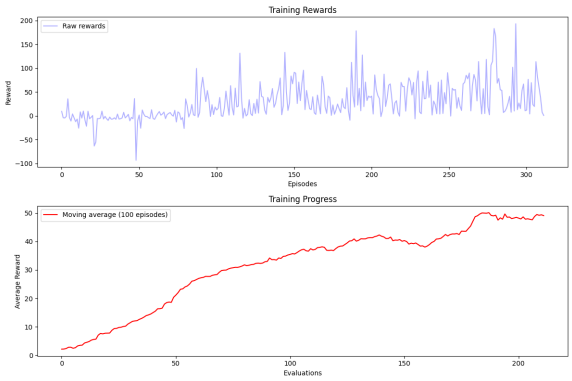

Fig. 4. Graf 2. fázy trénovania

Potom čo náš agent dokázal obstojne prejsť po celej mape bez závážných chýb a kolízií sme pristúpili k tretej fáze učenia. Táto fáza spočívala v postupnom pridávaní premávky do prostredia. Hlavnou úlohou tejto fázy bolo naučiť agenta vyhýbať sa kolíziám s inými autami. Pri prvých pokusoch sme do premávky pridali aj jednostopové vozidlá. Pri sledovaní priebehu učenia sme narazili na problémom, ktorý spočíval v tom, že agent sa nedokázal naučiť rozoznať jednostopové vozidlá. Predpokladáme, že pri spracovaní týchto snímkov došlo k stratám a agent nedokázal jednoznačne určiť, že ide o vozidlo. Riešením bolo odstránenie jednostopových vozidiel. Agent sa následne dokázal naučiť vyhýbať sa kolíziám s inými vozidlami. Vyskytol sa však nový problém po pridaní premávky. Križovatky sú riadené svetelnou signalizáciou, ktorú náš agent zatiaľ neovládal. To viedlo ku kolíziám v križovatkách, ktorým sa agent nedokázal vyhnúť.

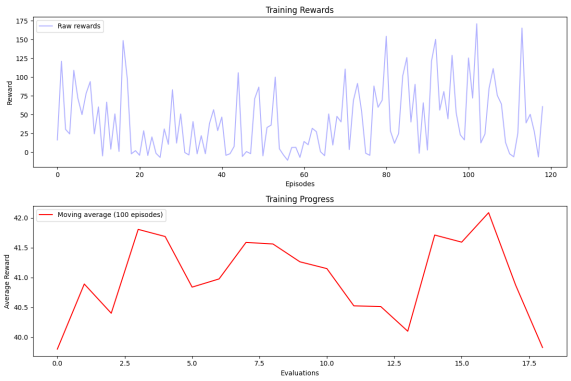

Fig. 5. Graf 3. fázy trénovania

V neskoršej fáze sme sa teda pokúsili o implementovanie rozoznania svetelnej signalizácie, pri ktorom sme narazili na ďalší závažný problém, kde nami zvolená segmentačná kamera nezobrazovala jednotlivé farby svetelnej signálizácie. Tieto farebné signalizácie sme sa snažili implementovať pomocou iných techník využitim API simulátora carla. Prvotným návrhom bolo zobraziť do pravého horného rohu farebný kruh najbližšieho semafora, ktorý má vplyv na jazdný pruh v ktorom sa agent nachádza. Toto naše prve riešenie, by z logického uhla pohľadu malo fungovať, z nejakého uhla pohľadu nefungovalo. Po zamyslení sa nad týmto problémom a nad tým čo už agent ovláda sme došli k riešeniu, že v prípade zelenej farby na semafore agentovi nemusíme signalizovať nič. Postačujúca bude pôvodná reward funkcia, ktorá ho bude povzbudzovať prejsť cez križovatku. Pri červenej farbe na semafore nám stačí zobraziť na kameru kruh rovnakej farby akej sú vozidlá. Agent teda bude mať pocit, že pred ním stojí autom aj keď v realite tomu tak nie je. Toto riešenie sa ukázalo ako efektívne. Agent sa dokázal naučiť jazdiť v premávke bezpečne, čo bolo aj naším cieľom.

Fig. 6. Graf dotrénovania

V nasledujúcej tabuľke môžete vidieť počet kolízií a vybočení z cesty počas trénovania modelu. Do tabuľky sme zapísali aj približný čas trénovania konkrétnych fáz. Celkovú dobu trénovania sme však určili na dobu dvoch dní, nakoľko pri trénovaní sme sa viackrát vrátili k predošlím checkpointom a optimalizovali reward funkciu.

TABLE II
VYSLEDNÉ HODNOTY MODELU DQN

Počet Kolízii	634
Počet OffRoad	457
Dlžka trenovania fáza č.1	2 hodiny 15 minút
Dlžka trenovania fáza č.2	3 hodiny 20 minút
Dlžka trenovania fáza č.3	1 hodina 40 minút

Na trénovanie nášho modelu sme využili knižnicu stable_baseline, ktorá nám umožnuje implementáciu modelu DQN, pri ktorej sme využili nasledujúce parametre, ktoré su znázornené v tab.č.3.

TABLE III
HYPERPARAMETRE MODELU DQN

policy	CnnPolicy
learning_rate	0.00001
buffer_size	20 000
learning_starts	1000
batch_size	32
gamma	0.995
train_freq	4
target_update_interval	500
exploration_fraction	0.4
exploration_initial_eps	1.0
exploration_final_eps	0.0

B. Model PPO

Pre tréning modelu PPO sme zvolili spojitý akčný priesor, nakoľko sme chceli dosiahnuť plynulejšiu jazdu našim agentom. Prvotne sme zvolili hodnoty otáčania volantu v rozsahu $[-1, 1]$, a pre reguláciu plynu v rozsahu $[0, 1]$.

Ako reward funkciu sme sa využili pôvodnú funkciu, ktorú sme vytvorili pre model DQN. Tá nevyhovala tomuto modelu, teda nedosahovala očakávajúce výsledky. Agent sa naučil točiť do kruhu, alebo vo vysokej rýchlosti nabúral. Napriek menším úpravam v najlepšom prípade využíval krajné hodnoty pre ovládanie smeru, a tým sa pohyboval od stredu po kraj cesty, teda robil slalom.

Napokon sme sa rozhodli znížiť rozsah akčného priestoru pre otáčanie volantu na $[-0.7, 0.7]$ a miernu úpravu reward funkcie. Hlavné zmeny tkveli v penalizovaní za príliš prudké otáčanie volantu. Tu chceme poukázať na to, že 2 z 3 študentov, pracujúcich na tomto zadaní, nedokázali PPO agenta naučiť ani len využívať spojitý priestor. Po mnohých neúspochoch sme sa pokúsili vrátiť k diskrétnemu akčnému priestoru. To sa ukázalo ako horšie riešenie oproti spojitému akčnému priestoru.

Mysleli sme si že samotný tréning agenta PPO by mohol postupovať rovnakým spôsobom, ako pri predošlom trénovaní agenta DQN. Prvá fáza tréningu agenta PPO, ktorá bola zameraná na princípy vnímania okolia, rovnako ako aj na udržiavanie agenta na vozovke v správnom jazdnom pruhu a vyhýbanie sa okolitým objektom, ukázala na mnoho problémov, ktoré sa nám nepodarilo vyriešiť.

Hlavným problémom boli križovatky tvaru "T" pri ktorých bol agent na vedľajšej ceste. Agent sa nevedel rozhodnúť ktorým smerom postupovať čo viedlo k neželanej kolízii. Ďalším problémom je to, že agent občasne koliduje do okolitých objektov na chodníku v spomínaných križovatkách, ale občasne aj v zákrutách alebo na rovných cestách. Dotrénovávanie agenta postupne viedlo k jeho pretrénovaniu, teda k tomu, že hneď po začiatku epizódy nebol schopný v pokračovaní po rovnej ceste a vybočil do protismeru. Pokúsili sme sa teda pokračovať do ďalšej fázy a pridali sme dopravu. Predpokladali sme, že sa agent prispôsobí novým zmenám prostredia.

Výsledky však ukázali na to, že agent sa neprispôsobil doprave. V skorších fázach dodatočného trénovania agent ignoroval vozidlá, ktoré sa nachádzali pred ním, a kolidoval s nimi. Postupom dalšieho tréningu bola snaha o obídenie vozidiel, čo viedlo k ukončeniu epozódy kvôli prechodu do protismeru alebo mimo vozovky. Ďalšie trénovanie viedlo k postupnej strate doposial dosiahnutých znalostí.

Na nasledujúcom grafe môžete vidieť vzorec, ktorý sa počas trénovania PPO agenta opakoval veľmi často. Na grafe je viditeľný nárast reward hodnôt a následný pokles reward hodnôt, čo je dôkazom pretrénovania modelu.

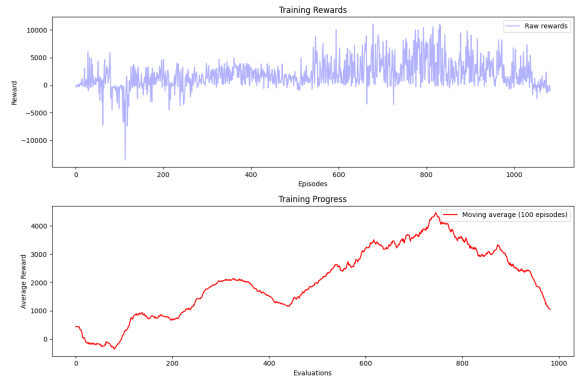

Fig. 7. Graf trénovania bez premávky

Na obrázku č.8 je zobrazený priebeh dotrénovania agenta

v premávke. Z grafu vyplýva, že agent sa nenaučil bezpečne jazdiť v premávke.

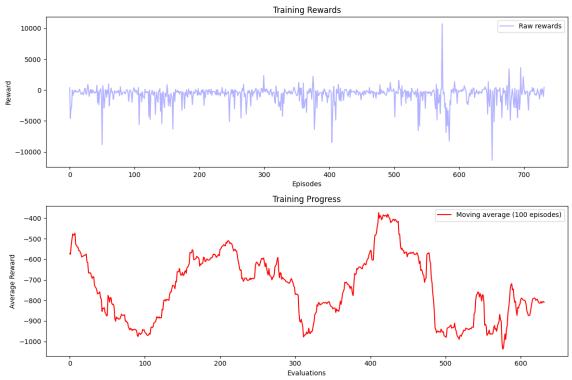

Fig. 8. Graf trénovania s premávkou

V nasledujúcej tabuľke môžete vidieť počet kolízií a vybočení z cesty pri trénovaní. V tabuľke nájdete aj približný čas trénovania modelu vo fázach, ktoré sme vyskúšali. Celkovú dobu trénovania modelu, bez ohľadu na množstvo iných pokusov, ktoré sme tu radšej ani nespomínali, by sme určili na 3 dni.

TABLE IV
VYSLEDNÉ HODNOTY MODELU PPO

Počet Kolízii	482
Počet OffRoad	757
Dlžka trenovania fáza č.1	5 hodiny 47 minút
Dlžka trenovania fáza č.2	3 hodiny 21 minút

Na trénovanie agenta sme využili knižnicu stable_baseline, ktorá nám umožnuje implementáciu modelu PPO, pri ktorej sme využili nasledujúce parametre, ktoré su znázornené v tab.č.5.

TABLE V
HYPERPARAMETRE MODELU PPO

policy	CnnPolicy
learning_rate	0.0001
n_steps	2048
batch_size	32
gamma	0.99
gae_lambda	0.95
clip_range	0.2
ent_coef	0.01
vf_coef	1.0
use_sde	True

III. ZÁVER

V rámci našej práce sa nám podarilo úspešne natrénovať agenta DQN, ktorý sa dokáže bezpečne pohybovať v premávke.

Agenta PPO sa nám podarilo naučiť pohybovať v scéne bez premávky s občasnými kolíziamy, avšak na zaradenie do premávky nebol vhodný.